

Competitive Programming

Section 23: Knapsack And Subset DP

Competitive Programming Course

2026-07-10

Table of contents

1	Why knapsack matters	3
2	Recognition	4
3	The two-state version	5
4	Compressing to one dimension	6
5	0/1 knapsack: loop backward	7
6	Complete knapsack: loop forward	8
7	Subset-sum feasibility	9
8	Visual comparison	10
9	Worked example: 0/1 knapsack	11
10	Worked example: possible sums	12
11	Local exercises	13
11.1	A. Budget Selection	13
11.2	B. Unlimited Training	13
11.3	C. Possible Sums	13
12	C++ implementation notes	14
13	Python implementation notes	15
14	Proof habits	16
15	Common mistakes	17
16	Local judge workflow	18
17	Submission checklist	19
18	Exercises	20

1 Why knapsack matters

Knapsack is the first DP family where the index is not time or position. The state is usually a capacity, sum, or budget.

Typical statement:

```
There are items.  
Each item has a cost/weight.  
Each item has a value, or contributes to a reachable sum.  
Choose some items under a limit.
```

Brute force tries every subset:

```
2n choices
```

Knapsack DP replaces “which exact subset did I choose?” with a smaller summary:

```
what total capacity/sum have I used?
```

2 Recognition

Use knapsack or subset DP when the statement has:

Statement signal	Likely DP
choose each item at most once	0/1 knapsack
choose items under total weight/cost W	capacity DP
item types can be used unlimited times	complete knapsack
print all reachable sums	subset-sum feasibility
constraints include small total sum or small capacity	DP over capacity/sum

The capacity must be small enough. If $W = 10^9$, direct $O(nW)$ DP is not viable, and you may need a different state such as total value. That variant is introduced in the practice queue, not required by the local exercises.

3 The two-state version

For 0/1 knapsack, the clearest state is:

```
dp[i][cap] = best value using only the first i items and capacity cap
```

Transition for item i with cost c and value v :

```
skip item i:  
    dp[i][cap] = dp[i-1][cap]  
  
take item i, if cap >= c:  
    dp[i][cap] = dp[i-1][cap-c] + v
```

So:

```
dp[i][cap] = max(dp[i-1][cap], dp[i-1][cap-c] + v)
```

This is often too much memory, but it is the safest way to derive the recurrence.

4 Compressing to one dimension

Since row i only depends on row $i-1$, we can store one array:

```
dp[cap] = best value for this capacity after processing some prefix of items
```

Now loop direction becomes critical.

5 0/1 knapsack: loop backward

Each item can be used at most once.

For an item (cost = c , value = v):

```
for cap from W down to c:  
    dp[cap] = max(dp[cap], dp[cap-c] + v)
```

Why backward?

When updating $dp[cap]$, the value $dp[cap-c]$ must still mean “before this item was processed”. If we loop forward, $dp[cap-c]$ may already include the same item, which uses it twice.

Bad forward trace:

```
one item: cost 3, value 10  
W = 6  
  
forward loop:  
cap = 3 -> dp[3] = 10  
cap = 6 -> dp[6] = dp[3] + 10 = 20    wrong: same item used twice
```

Backward loop:

```
cap = 6 reads old dp[3] = 0  
cap = 3 writes dp[3] = 10
```

Correct.

6 Complete knapsack: loop forward

In complete knapsack, each item type can be used any number of times.

For an item (cost = c , value = v):

```
for cap from c to W:  
    dp[cap] = max(dp[cap], dp[cap-c] + v)
```

Why forward?

Now reusing the same item is allowed. When $dp[cap-c]$ has already improved using the current item, that represents taking one more copy.

Trace:

```
one item type: cost 3, value 5  
W = 7  
  
cap = 3 -> dp[3] = 5  
cap = 6 -> dp[6] = dp[3] + 5 = 10
```

That is correct because two copies are allowed.

7 Subset-sum feasibility

Subset-sum DP asks which sums are reachable.

State:

```
possible[s] = true if some processed subset has sum s
```

Base:

```
possible[0] = true
```

For a number x , if sum s was reachable before this number, then $s+x$ becomes reachable after using it:

```
possible[s+x] = true
```

Because each number can be used at most once, loop backward:

```
for s from current_sum down to 0:
    if possible[s]:
        possible[s+x] = true
```

8 Visual comparison

The same update formula can mean different things depending on loop direction:

Problem	Item reuse	Capacity loop
0/1 knapsack	at most once	descending
subset sum	at most once	descending
complete knapsack	unlimited	ascending

Memorize the reason, not just the direction.

9 Worked example: 0/1 knapsack

Items:

```
(cost 3, value 4)
(cost 4, value 5)
(cost 2, value 3)
W = 7
```

Start:

```
cap: 0 1 2 3 4 5 6 7
dp:  0 0 0 0 0 0 0 0
```

After (3,4):

```
dp:  0 0 0 4 4 4 4 4
```

After (4,5):

```
dp:  0 0 0 4 5 5 5 9
```

After (2,3):

```
dp:  0 0 3 4 5 7 8 9
```

Best value with capacity at most 7 is 9.

10 Worked example: possible sums

Numbers:

```
4 2 5 2
```

Reachable sums evolve:

```
start:      {0}
after 4:     {0,4}
after 2:     {0,2,4,6}
after 5:     {0,2,4,5,6,7,9,11}
after 2:     {0,2,4,5,6,7,8,9,11,13}
```

Positive sums:

```
2 4 5 6 7 8 9 11 13
```

11 Local exercises

11.1 A. Budget Selection

Each resource can be chosen at most once.

State:

```
dp[cap] = best value reachable with capacity at most cap after processed items
```

Loop:

```
for cap from W down to cost
```

11.2 B. Unlimited Training

Each drill can be repeated any number of times.

State:

```
dp[cap] = best value reachable with capacity at most cap after processed types
```

Loop:

```
for cap from cost up to W
```

11.3 C. Possible Sums

Each number can be used at most once.

State:

```
possible[s] = whether sum s is reachable
```

Loop:

```
for s downward
```

12 C++ implementation notes

Use `long long` for values:

```
vector<long long> dp(W + 1, 0);
```

For subset feasibility, `vector<char>` is compact and simple:

```
vector<char> possible(total + 1, false);
```

Avoid `vector<bool>` while learning. It is a specialized bitset-like container with surprising reference behavior.

13 Python implementation notes

Capacity arrays are ordinary lists:

```
dp = [0] * (W + 1)
possible = [False] * (total + 1)
```

Backward loops:

```
for cap in range(W, cost - 1, -1):
```

Forward loops:

```
for cap in range(cost, W + 1):
```

14 Proof habits

The proof usually tracks an invariant after each item:

```
After processing the first i items/types, dp[cap] is the best value achievable  
under the rules using only those processed items/types.
```

Then justify loop direction:

```
Backward means the current item is not visible to itself.  
Forward means the current item is allowed to build on itself.
```

15 Common mistakes

```
Using forward loops for 0/1 knapsack and accidentally reusing an item.  
Using backward loops for complete knapsack and accidentally allowing only one copy.  
Forgetting possible[0] = true in subset sum.  
Printing dp[W] when the array stores exact capacity and smaller capacity is allowed.  
Using int for values up to 1e9 over many items.  
Trying  $O(nW)$  when  $W$  is too large.
```

16 Local judge workflow

Run this section:

```
CP_TARGET=solution python3 sections/23_knapsack_subset_dp/check.py
```

Run everything:

```
CP_TARGET=solution python3 tools/check_all.py
```

17 Submission checklist

Before submitting:

```
Is each item used once or unlimited times?  
Does my loop direction match that rule?  
What does dp[cap] mean after each item?  
Is the capacity/sum small enough for  $O(nW)$ ?  
Do values need long long?  
For subset sum, did I initialize sum 0 as reachable?
```

18 Exercises

Use A for 0/1 capacity direction, B for complete-knapsack direction, and C for boolean subset reachability. Test budget zero, an item larger than the capacity, and repeated values.